

FORMATION EMBARQUÉE

TP5 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 1 : socle STM32 (horloges, UART, GPIO) (2 h)

En-tête pédagogique

Objectifs	Créer un projet CubeIDE ; comprendre le clock tree ; rediriger printf ; heartbeat non bloquant ; bouton par interruption
Prérequis	Module 10 + TD 10 faits ; TP 1 (l'esprit non bloquant)
Outils	STM32CubeIDE ; Nucleo-F411RE (adapter si autre carte)
Livrable	Bannière série + heartbeat + bouton pris en compte

Déroulé minuté

0:00-0:20 — Projet et clock tree

1. **File → New → STM32 Project** → Board Selector → Nucleo-F411RE → initialiser les périphériques par défaut : **oui**.
2. Onglet **Clock Configuration** : vise **100 MHz** sur HCLK (tape 100 dans la case, CubeMX résout la PLL). Note dans ton journal les valeurs M/N/P choisies, et **vérifie que APB1 ≤ 50 MHz** (au-delà, certains périphériques ne suivent pas). Comprendre cet arbre, c'est comprendre d'où vient la fréquence de chaque timer/bus (§3 du module 10).

0:20-0:50 — Redirection de printf

Génère le code (Alt+K). Dans **main.c**, entre les balises USER CODE (rappel : tout code hors de ces balises est **écrasé** à la régénération) :

```
/* USER CODE BEGIN 0 */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, 10);
    return ch;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
printf("=== Station meteo STM32 ===\r\n");
printf("Build : %s %s\r\n", __DATE__, __TIME__); // savoir QUEL firmware tourne
/* USER CODE END 2 */
```

Ouvre un terminal série (115200) sur le port ST-Link virtuel (**/dev/ttyACM0** ou COMx).  **Point de contrôle 1** : la bannière s'affiche au reset.

Astuce : la bannière avec **__DATE__ / __TIME__** est un réflexe pro — en debug, elle te dit si la carte

exécute bien ta dernière compilation.

0:50-1:20 — Heartbeat non bloquant

La LED LD2 clignote à 1 Hz sans `HAL_Delay` (le socle doit déjà être non bloquant, comme le TP 1) :

```
/* USER CODE BEGIN WHILE */
uint32_t t_led = 0;
while (1) {
    uint32_t now = HAL_GetTick();           // ms depuis le démarrage
    if (now - t_led >= 500) {
        t_led = now;
        HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    }
}
/* USER CODE END WHILE */
}
```

C'est le `millis()` d'Arduino, version STM32. Même motif de soustraction robuste au débordement.

1:20-1:50 — Bouton par interruption EXTI

Dans CubeMX : la broche du bouton bleu (PC13, `B1`) est déjà en `GPIO_EXTI`. Vérifie que l'interruption `EXTI15_10` est activée dans l'onglet NVIC. Régénère, puis :

```
/* USER CODE BEGIN PV */
volatile uint8_t verbeux = 1;           // volatile : modifié en ISR
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == B1_Pin) {
        verbeux = !verbeux;           // ISR courte : un drapeau, point
    }
}
/* USER CODE END 0 */
```

Utilise `verbeux` dans la boucle pour activer/couper des logs. Les règles d'ISR du module 00 s'appliquent : courte, `volatile` sur la variable partagée.

✅ **Point de contrôle 2** : appui bouton → la verbosité change (logs qui apparaissent/disparaissent), heartbeat toujours régulier.

1:50-2:00 — Structure du dépôt + commit

Crée `Core/Src/drivers/` (les drivers des séances suivantes y iront). Init Git sur le projet, `.gitignore` pour `Debug/` et les artefacts. Commit : `git commit -m "TP5 seance 1 : socle horloges + printf + heartbeat + bouton"`.

Journal : note les valeurs M/N/P de ta PLL et la fréquence de chaque bus (APB1, APB2). Tu en auras besoin pour calculer les prescalers des timers (séance 3).

Erreurs fréquentes

Symptôme	Cause	Remède
Rien au terminal	mauvais port, vitesse $\neq$ 115200, printf non redirigé	vérifier <code>__io_putchar</code> et le port ST-Link
Code disparu après régénération CubeMX	code hors des balises USER CODE	toujours entre BEGIN/END
Bouton sans effet	EXTI non activé dans NVIC, ou mauvais pin	vérifier onglet NVIC
Heartbeat irrégulier	un HAL_Delay traîne dans la boucle	interdit : utiliser HAL_GetTick
Warning float dans printf	<code>%f</code> sans l'option « use float with printf »	cocher l'option projet, ou éviter %f

Travail à la maison (30 min)

Ajoute un **anti-rebond logiciel** au bouton : le callback EXTI peut être appelé plusieurs fois par appui (rebonds). Enregistre `HAL_GetTick()` dans l'ISR et ignore les appuis à moins de 200 ms d'écart. Constate la différence.

➡ Fiche suivante : [Séance 2 — Driver BME280 maison](#)